CSC-239 *Java Programming*

Chapter 13: Abstract Classes and Interfaces

This lecture is being recorded.

October 27, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

- BHCC Tutoring:

  https://bhcc.edu/tutoring/

Quick Review of Concepts

- **Class**: Programmer-defined datatype representing some kind of object.
- **Object**: an *Object* is an instance of a *Class*.
- **Inheritance**: one class can be a specialization of another class.
- The "**IsA**" relationship:

  An object of a **subclass** Is also an object of the **superclass**.

Inheritance Chain and the **ISA** relationship

Suppose we have three classes --

Person, Employee, Faculty:

Employee extends **Person**.

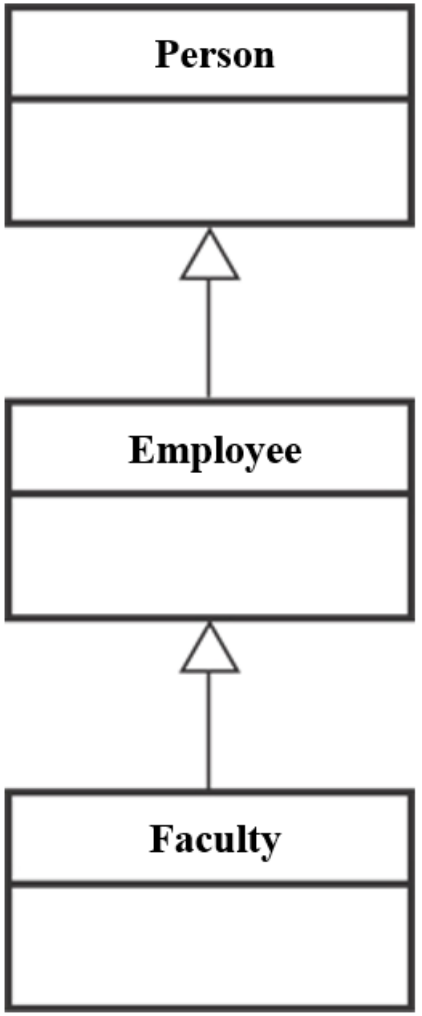
Faculty extends **Employee**.

A **Faculty** object is an **Employee** object.

An **Employee** object is a **Person** object.

Therefore:

A **Faculty** object is a **Person** object.



Superclasses and Subclasses (1)

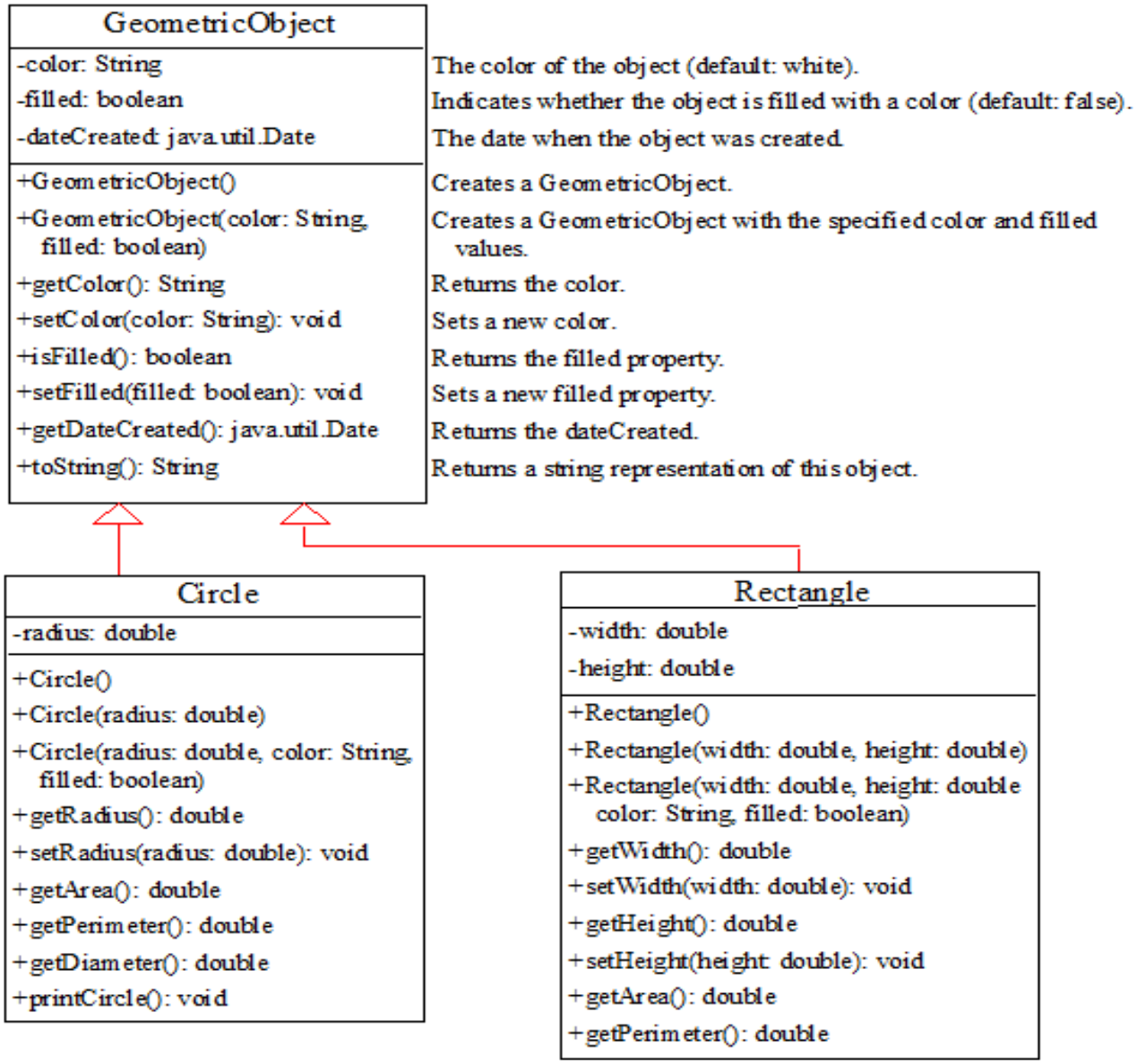
- Inheritance involves one class being a *subclass* of another class.
- The UML diagram on the next slide illustrates an inheritance relationship among the

  GeometricObject

  Circle, and

  Rectangle classes.
- These classes are described in Chapter 11 of the textbook.
- The hollow arrow showing inheritance in a UML diagram always points to the superclass.

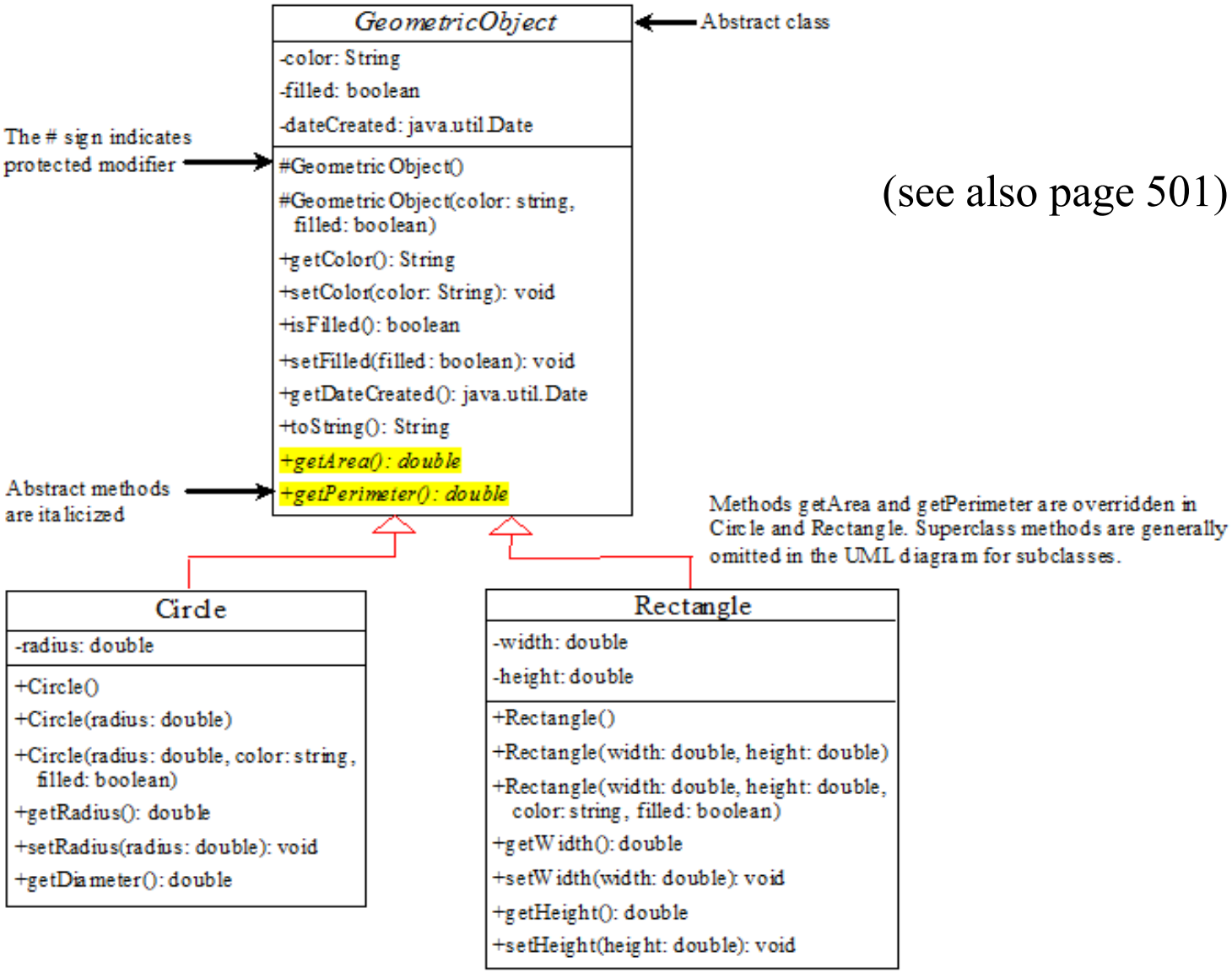
Superclasses and Subclasses (2)



Abstract vs. Concrete Classes

- An *Abstract Class* cannot be used to create objects directly (it cannot be "instantiated").
- A sub-class of an abstract class may be a "concrete" class, which *can* be instantiated.
- An abstract class *may* contain abstract methods, which are implemented in concrete sub-classes.

Example: Abstract Classes and Methods



(see also page 501)

Abstract Classes and Methods -- Syntax

```
public abstract class GeometricObject {
. . .


 /** Abstract method getArea */
  public abstract double getArea();

  /** Abstract method getPerimeter */
  public abstract double getPerimeter();
}
```

Abstract Method in Abstract Class

- Any abstract method(s) defined in the abstract parent class, must be implemented in the nonabstract child class, even if they are not used.

Abstract Class Constructors

- Constructor(s) in the abstract class are invoked (called) by constructor(s) in the nonabstract subclass.
- For example, the constructors of GeometricObject are invoked by the constructors in Circle and Rectangle.

Abstract Methods

- An abstract class *may* or *may not* contain abstract method(s).
- Abstract methods can *only* exist in an abstract class.

TestGeometricObject.java

- This sample program is shown on pages 502-503 of the text book.
- The source code is also available on *Moodle* in the "Sample Code" section.
- The program creates a **Circle** object and a **Rectangle** object, and then determines whether the two objects have the same area.

Abstract Subclass, Concrete Superclass

- It is possible for a parent class to be Concrete (nonabstract), while its child class is abstract.
- For example, the Object class is concrete, but its subclasses, such as GeometricObject, may be abstract.

Abstract Class as a type

- An abstract class can be used as a data type.
- For example:

```
GeometricObject[] geo =
    new GeometricObject[10];
```

- Recall that an array of objects is actually an array of *reference variables* to that class.
  - The default value of each array element is null.

What is an interface?

An interface is a "classlike" construct that may contain:

- constants
- abstract methods
- default methods
- static methods
- nested types

(For now, we are going to focus only on **constants** and **abstract methods**.)

Why is an interface useful?

- An *interface* is similar to an *abstract class*, but the intent of an interface is to specify common behavior for objects of different classes.
- If you have studied the C++ programming language, you may have noticed that **Java** does not support multiple inheritance in the same way that C++ does.

  You can think of the Java ***interface*** as being conceptually similar to multiple inheritance in C++.

Interface is a Special Class

- Compiled into a separate **`.class`** file.
- Like an *abstract class*, it cannot be directly instantiated: we cannot create an object of an interface.
- Interface Syntax:

```
public interface InterfaceName {
    constant declarations;
    method signatures;
}
```

Example: The Comparable Interface

```
// This interface is defined in
// the java.lang package

package java.lang;

public interface Comparable<E> {
   public int compareTo(E o);
}
```

The `compareTo()` method

- Compares the current object (**`this`**) with the object passed as a parameter:

```
int compareTo( T o)
```

- Returns an integer, which is
  - *negative* if **`this`** object is "less than" the object passed as the parameter
  - *zero* if **`this`** object is "equal to" the object passed as the paramter
  - *positive* if **`this`** object is "greater than" the object passed as the parameter

String and Date implement Comparable

```
public class String extends Object
    implements Comparable<String> {
  // class body omitted

  @Override
  public int compareTo(String o) {
    // Implementation omitted
  }
}
```

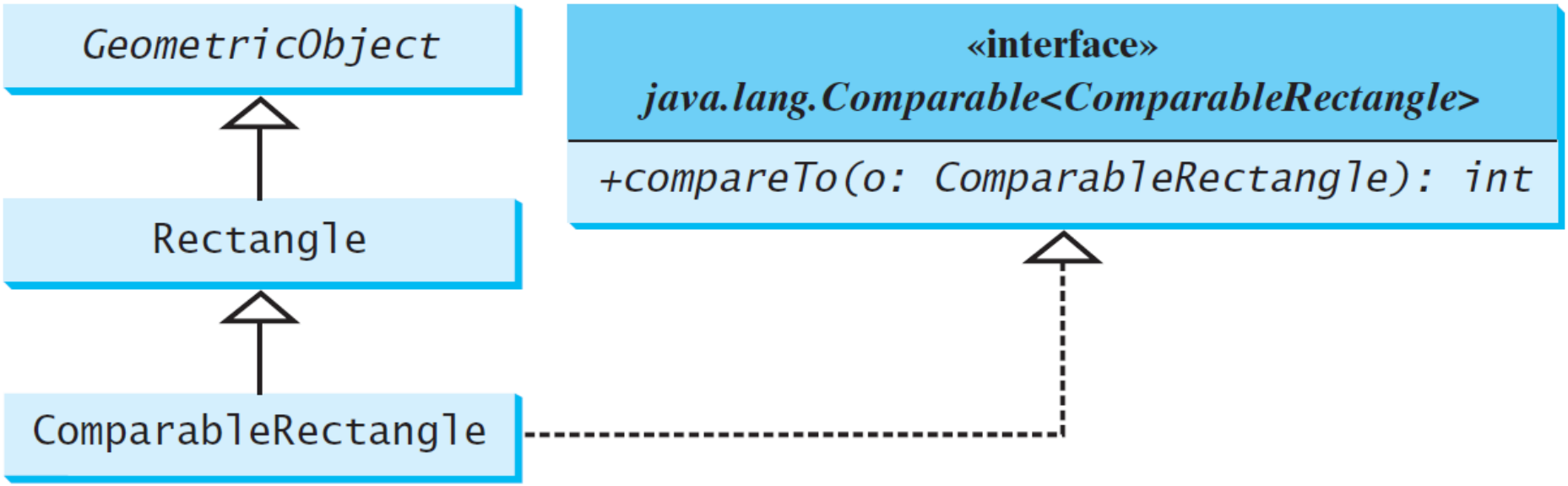
```
public class Date extends Object
    implements Comparable<Date> {
  // class body omitted

  @Override
  public int compareTo(Date o) {
    // Implementation omitted
  }
}
```

```
System.out.println("ABC".compareTo("ABE"));

java.util.Date date1 =
              new java.util.Date(2013, 1, 1);
java.util.Date date2 =
              new java.util.Date(2012, 1, 1);
System.out.println(date1.compareTo(date2));
```

Defining Classes to Implement Comparable



ComparableRectangle.java

- A subclass of the Rectangle class
  - Implements the "Comparable" interface:

```
public class ComparableRectangle
 extends Rectangle
 implements Comparable<ComparableRectangle>
{   . . .  }
```

- Must include a method:

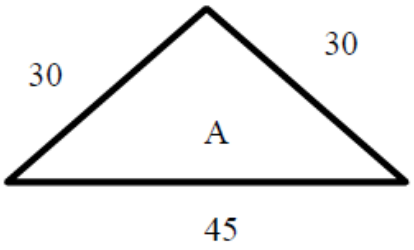
```
public int compareTo(ComparableRectangle o)
{ . . . }
```

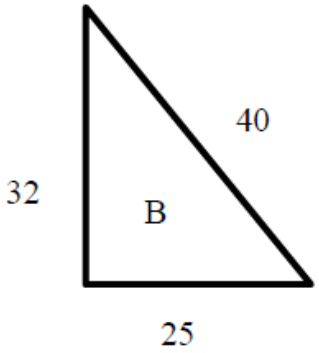
SortRectangles.java

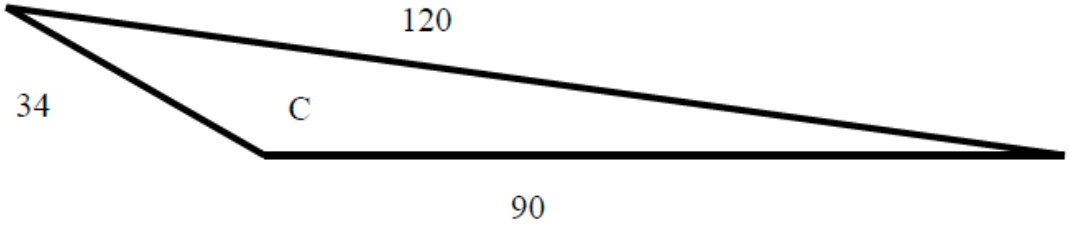
- Once you have a **ComparableRectangle** class, this enables a program to sort an array of such objects

Lab13a: A Triangle Class

- A **Triangle** ISA **GeometricObject**
- Some sample Triangles:







Lab13a: Abstract classes

- Use the **TestGeometricObject** sample code (on *Moodle*) as a guide.
- The solution must have two new classes:

  Lab13a: This is a public class that contains a **main** method. The main method must ask the user to enter details about the Triangle object, create the object, and output details about the object.

  Triangle: This class is a subclass of the GeometricObject class.

Java Programming, Project 2

- In Project 2, you will implement a text-based Java program that simulates the processing of credit card transactions.
- Project 2 also involves reading and writing data in actual (text-based) data files.
- Project 2 is due on November 17.